

Face Recognition



Team Members

Name	ID
Hager Ashraf Mohamed Melook	21011505
Nouran Ashraf Yousef	21011492
Rana Mohamed Ali Attia	21010528

Overview

We intend to perform face recognition. Face recognition means that for a given image you can tell the subject id. Our database of subjects is very simple. It has 40 subjects. Below we will show the needed steps to achieve the goal of the assignment.

Dataset Overview

- ORL dataset: 40 subjects, 10 grayscale images per subject (size: 92x 112).
- Training/Test Split: Odd rows (training), even rows(testing).

Implementation Details

PCA

Purpose:

Dimensionality reduction: compressing face data (10,304) features into fewer components while retaining specific variance (threshold = 0.8,0.85,0.9,0.95).

Steps:

Centering Data: by subtracting the mean vector from input.

Covariance Matrix:

$$COV = \frac{1}{n} \times (X_{Centered}^T \times X_{Centered})$$

Eigen Decomposition:

- solving eigenvectors/values
- Normalization
- Sorting by descending eigenvalues.

Thresholding: retain minimum components needed to meet variance

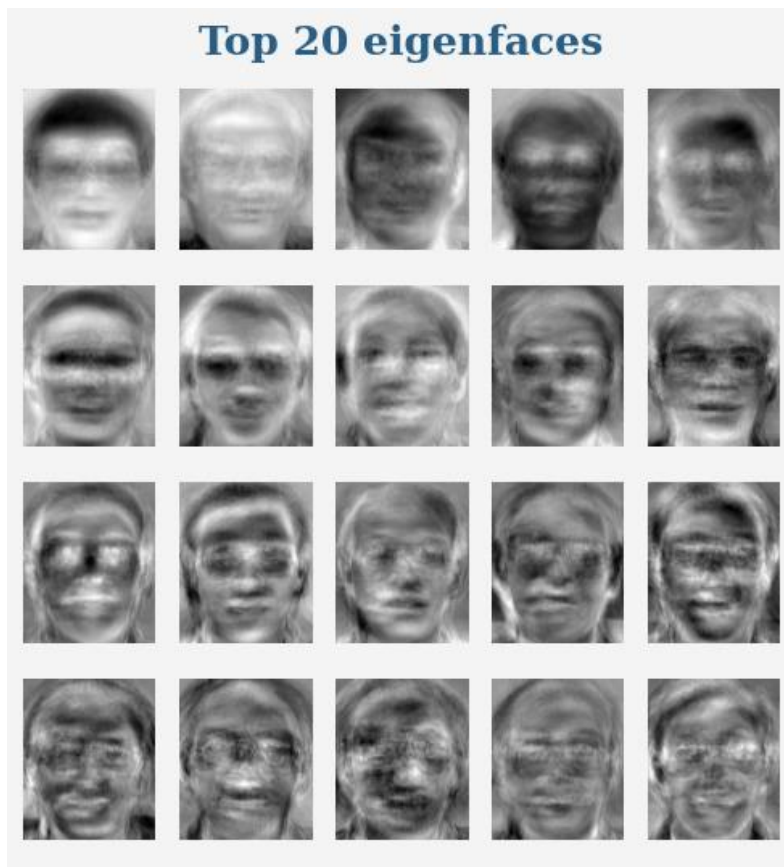
Optimizations:

Reuse Components: compute once for highest alpha and use subsets for lower thresholds:

Speed: 0.08 sec computation time (cached).

Visualizations:

- **Eigenfaces:**



- **Reconstructions:**

Best reconstruction quality at $\alpha = 0.95$

Original vs Reconstructed (threshold = 0.95)

Original



Reconstructed



Trade-offs:

- Lower alpha: fewer components → **Faster computation** but **blurry reconstructions**.
- Higher alpha: more components → **Accurate reconstructions** but **higher dimensionality**.

Unsupervised Clustering

1. K-Means Clustering

Purpose: partition a dataset into distinct groups (clusters) based on similarity, without using labeled data.

Also, it is useful if the number of labels/objects are large.

Steps:

1. **Initialization of cluster centroids:** using the kmeans++ idea to initialize the cluster centroids:-
 - Choose the first one randomly from the 200 samples.
 - repeatedly, choose the next centroids by get the min distance from each sample to centroid and use them as probability so the already chosen centroids will have probability 0.

Repeatedly until converge:

2. **Assignment:** assign the sample to cluster label whose centroid is the nearest to this sample.
3. **Update Centroids:** re-compute the cluster centroids to be the center of gravity of the cluster by assigning it to the mean of the assigned samples to this cluster.

Note: if the cluster has no samples, re-initialize it with a random sample.

Convergence Criteria:

Use `allclose()` with:

- **`rtol=1e-5`**: relative tolerance (proportional to the size of the values).
- **`atol=1e-8`**: absolute tolerance (minimum difference accepted even for very small values).

```
numpy.allclose(a, b, rtol=1e-05, atol=1e-08)
```

It returns True if: $|a - b| \leq atol + rtol \times |b|$

For each corresponding element in arrays `a` and `b`

Evaluation: calculate the accuracy according to the get the majority sample Label that belong to see the cluster and assign this samples the majority Label and the accuracy is the ratio of how much correctly classified samples to total number of samples.

Analysis:

There is a relation between the `k` and the accuracy

- A small `k` (e.g., 20) may not capture the underlying structure well (underfitting).
- A larger `k` (e.g., 60) allows more fine-grained clustering, which can better adapt to complex data distributions — especially after dimensionality reduction (PCA).
- At `k = 20`: Low accuracy (~0.41)
- At `k = 40`: Improved accuracy (~0.65–0.695)
- At `k = 60`: Best accuracy (~0.8–0.825)

so increasing `k` gives better clustering granularity, which results in better classification accuracy.

There is a relation between the `alpha` and accuracy:-

- At `alpha = 0.8`: Retains 80% of variance → more dimensionality reduction → less information, but simpler clusters.

This will result in an underfitting model which won't capture the complex details of data.

- At `alpha = 0.95`: Retains 95% of variance → less reduction → more detailed data, but can introduce noise or redundancy.

This will result in an overfitting model which cares about small changes and is very sensitive to noise.

so the best accuracy will be at `alpha 0.9` (better accuracy compared to `alpha 0.85` as it capture more variation & details)

2. GMM Clustering

Steps:

Initialization:

- The weights (π) are initialized uniformly by dividing 1 by the number of clusters.
- The means are initialized by assigning them using the KMeans built-in algorithm.
- The covariance matrix is initialized as the identity matrix.

Expectation Step:

- The responsibility matrix is calculated using the specified rule.

$$r_{ik} = \frac{\pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(x_i | \mu_j, \Sigma_j)}$$

Maximization Step:

- The cluster means, covariances, and weights (π) are updated.

$$\begin{aligned}\pi_k &= \frac{1}{N} \sum_{i=1}^N r_k^{(i)} \\ \mu_k &= \frac{\sum_{i=1}^N r_k^{(i)} \cdot \mathbf{x}^{(i)}}{\sum_{i=1}^N r_k^{(i)}} \\ \Sigma_k &= \frac{1}{\sum_{i=1}^N r_k^{(i)}} \sum_{i=1}^N r_k^{(i)} (\mathbf{x}^{(i)} - \mu_k)(\mathbf{x}^{(i)} - \mu_k)^\top\end{aligned}$$

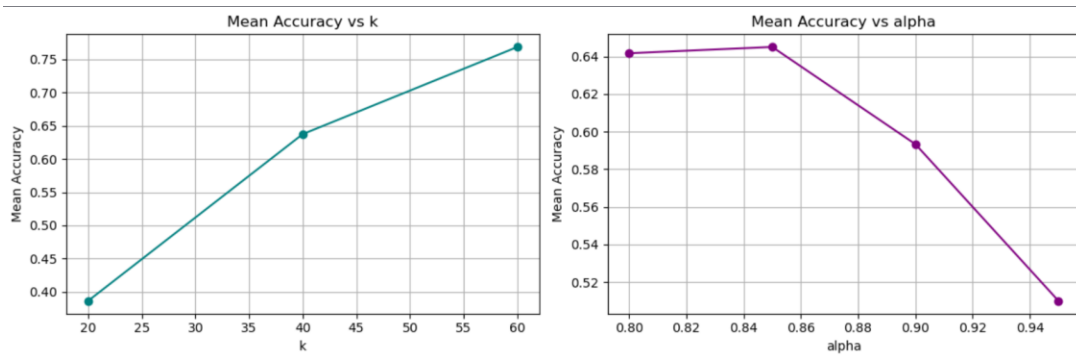
We then perform **EM** until the **covariances** and **means** converge, or until the **maximum iterations** are reached.

Optimizations:

- Logarithmic calculation to avoid numerical underflow.
- NumPy broadcasting for performance optimization.
- KMeans initialization for better cluster centroids.

Visualizations:

This shows the effect of changing the number of clusters and alpha on the GMM training accuracy.



Autoencoders

Purpose: nonlinear dimensionality reduction using convolutional layers to compress images into a latent space (R^{128}).

Architecture and Training

Encoder:

- **4 Conv2D layers** with stride = 2 (downsample $\rightarrow 256 \times 7 \times 6$)
- **Latent space:** fully connected layer ($256 \times 7 \times 6 \rightarrow 128$)

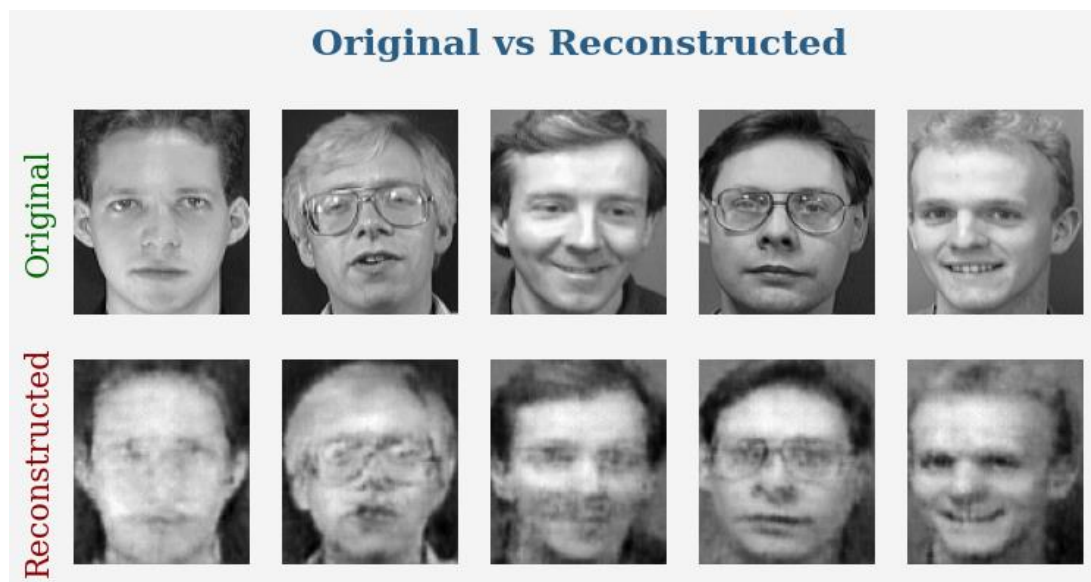
Decoder:

- **4 ConvTranspose2D layers** with stride = 2 (upsample $\rightarrow 1 \times 112 \times 92$).
- **Final activation:** Sigmoid function because pixel values normalized [0,1]

Training:

- **Loss:** MSE
- **Optimizer:** Adam with $lr = 1e-3$, weight decay = $1e-5$
- **Epochs:** 100

Reconstruction Quality:



Evaluation Metrics

- **Accuracy:** measures the proportion of correctly clustered samples relative to the total.
- **F1-Score:** balances precision which indicates cluster purity and recall which indicates cluster completeness.

$$F1 = \frac{2 \times \textit{precision} \times \textit{recall}}{\textit{precision} + \textit{recall}}$$

$$\textit{precision} = \frac{\textit{True positive}}{\textit{True positive} + \textit{False positive}}$$

$$\textit{recall} = \frac{\textit{True positive}}{\textit{True positive} + \textit{False negative}}$$

- **Confusion Matrix:** visualizes how clusters map to true classes and identifies misclassifications.

Results and Analysis

1. PCA results

Reduced Data Dimensions

Threshold	Train / Test size
0.8	(200,37)
0.85	(200,53)
0.9	(200,77)
0.95	(200,116)

2. Clustering Performance

2.1. K-Means

Threshold	Accuracy
0.8	0.636667
0.85	0.631667
0.9	0.64
0.95	0.613333

K	Accuracy
20	0.415
40	0.66875
60	0.8075

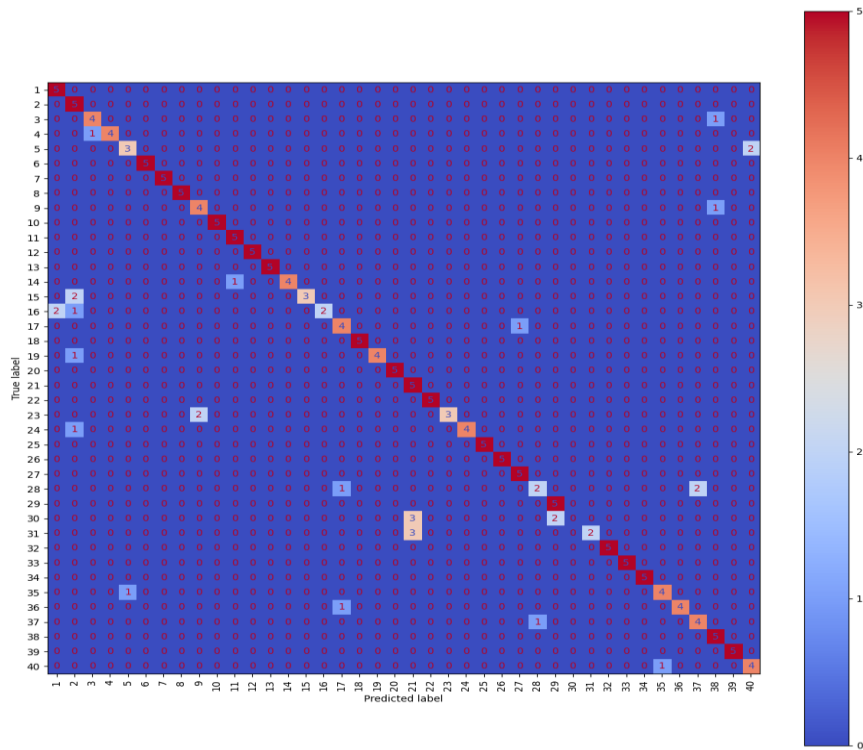
2.2. GMM

Train Accuracy			Test Accuracy		
k	alpha	accuracy	k	alpha	accuracy
20	0.8	0.39	20	0.8	0.37
40	0.8	0.725	40	0.8	0.645
60	0.8	0.81	60	0.8	0.77
20	0.85	0.42	20	0.85	0.39
40	0.85	0.695	40	0.85	0.66
60	0.85	0.82	60	0.85	0.775
20	0.9	0.385	20	0.9	0.36
40	0.9	0.605	40	0.9	0.5
60	0.9	0.79	60	0.9	0.7
20	0.95	0.35	20	0.95	0.335
40	0.95	0.525	40	0.95	0.505
60	0.95	0.655	60	0.95	0.565

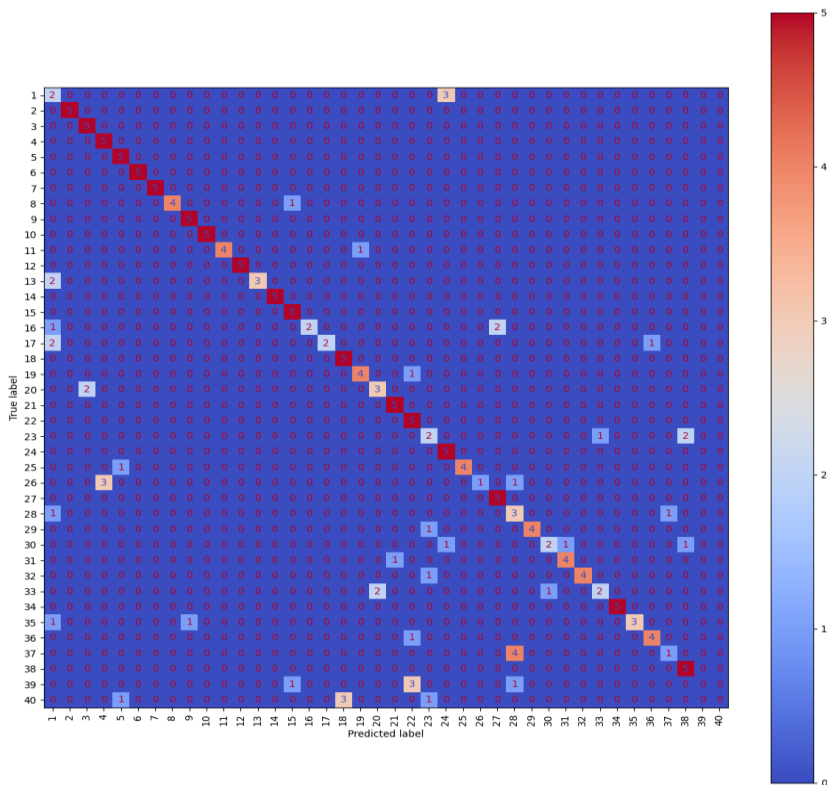
3. Test Set Evaluation

- For the KMeans:-
 - The best model is at k = 60 and alpha = 0.9
 - Accuracy score: 0.84
 - F1-Score: 0.8343226911976911

-Confusion matrix:



- For the GMM:
 - The best model is at $k = 60$ and $\alpha = 0.85$
 - Accuracy score: 0.74
 - F1-Score: 0.716519869019869
 - Confusion matrix:



Interpretation

- **K-Means** outperformed **GMM** across both accuracy and F1 Score.
- The **PCA threshold** played a role in feature compression:
 - 3.1. K-Means performed best with **90% retained variance**, suggesting that it benefited from richer feature representation.
 - 3.2. GMM peaked at **85% variance**, possibly due to overfitting at higher dimensions.

Conclusion

K-Means provided better clustering quality in this experiment, both in terms of classification **accuracy** and **F1 score**. Its simpler assumptions (hard clustering and spherical clusters) likely aligned better with the structure of the PCA-reduced data.

4. Autoencoder Result

Encoded Data: Training/Test shape = 200×128

Comparisons

Comparing PCA and Autoencoder:

PCA outperformed the Autoencoder, due to:

Deterministic vs Iterative Training:

- **PCA:** computes eigenvectors in closed form (no training iterations).
- **Autoencoder:** suffers from underfitting due to the small data size as 200 training samples are insufficient for a nonlinear model to generalize well.

Training Efficiency:

- **PCA:** computes in milliseconds after covariance matrix calculation.
- **Autoencoder:** slow training (backpropagation over 100 epochs).

Hyperparameters:

- **PCA:** single hyperparameter which is variance threshold. Easy to tune.
 - **Autoencoder:** Multiple hyperparameters (latent dimension = 128, learning rate, architecture including number of layers and their dimensions). So, it is very hard to tune, and mismatched latent space may harm the performance.
-

The results of clustering on reduced data of PCA vs Autoencoders:

Method	Accuracy	F1 score
K_means + PCA	0.84	0.83
K_means + Autoencoder	0.81	0.78
GMM + PCA	0.78	0.757
GMM + Autoencoder	0.68	0.655

Conclusion: the simple implementation of convolutional autoencoder may not learn sufficiently discriminative features for clustering. So, PCA's deterministic and closed form solution gives it the edge.